

Informe de desarrollo

1. Valoración del resultado final

- **¿El resultado cumple con lo esperado al inicio?** Sí, el proyecto final refleja fielmente los objetivos planteados en la fase de diseño. Hemos logrado una aplicación estable y funcional que cumple con el propósito inicial.
- **¿Habéis implementado toda la funcionalidad?** Se ha implementado el 100% de la funcionalidad core requerida.
- **¿Habéis añadido mejoras respecto a la idea inicial?** Sí, se han pulido detalles en la interfaz de usuario y se han optimizado ciertos procesos de backend para mejorar la experiencia de uso, superando ligeramente el alcance mínimo previsto.

2. Valoración del proceso de realización del proyecto

- **¿Qué tal ha ido la coordinación y reparto de tareas? ¿Todas las personas han colaborado por igual?** La coordinación ha sido excelente. El reparto de tareas se hizo de forma equitativa basándose en las preferencias y habilidades de cada uno. Aunque el ritmo de trabajo ha variado según la disponibilidad personal (algunos terminando sus tareas antes y otros más tarde), el compromiso ha sido total y todos han cumplido con sus responsabilidades.
- **¿Ha habido conflictos en el grupo? ¿Cómo los habéis solucionado?** No ha habido conflictos de carácter personal ni profesional. El "buen rollo" ha sido la tónica general, facilitando una comunicación fluida.
- **¿Habéis utilizado la hoja de cálculo de planificación para el seguimiento? ¿Habéis hecho un recuento de las horas reales? ¿Se acercan a las 50 h.?** Sí, se ha utilizado para monitorizar el avance. La media de dedicación se sitúa en torno a las 50-55 horas. El ligero exceso se debe principalmente a la curva de aprendizaje de algunas tecnologías específicas.
- **¿El desarrollo ha sido distribuido o ha habido periodos intensos?** Hubo un inicio constante, seguido de un periodo más relajado a mitad del semestre, y una intensificación final para integrar todas las partes y realizar el testing.
- **¿Qué tal ha ido la experiencia de uso de GitHub?** Técnicamente, ha sido una herramienta indispensable para el control de versiones. Sin embargo, a nivel de gestión de equipo, el análisis estadístico nos parece una métrica incompleta (ver apartado de Justificación de esfuerzos).
- **¿Cómo has resuelto las dudas y problemas?** Principalmente mediante consultas a la documentación oficial, apoyo mutuo entre los integrantes del equipo y uso de herramientas de IA para depuración.

- **¿Habéis utilizado alguna herramienta colaborativa?** Hemos utilizado Discord para reuniones rápidas y compartición de pantalla, y WhatsApp para la comunicación inmediata. La agilidad de estas herramientas ha sido clave para mantener el buen ambiente.

3. Identificación de cuestiones positivas y negativas

Cuestiones Positivas

1. **Ambiente de trabajo:** El respeto y la buena relación personal han evitado el estrés y han fomentado la ayuda mutua.
2. **Autonomía y Responsabilidad:** Cada integrante se hizo dueño de su parte, cumpliendo con los objetivos sin necesidad de supervisión constante.
3. **Resolución de problemas:** Capacidad de respuesta ante bugs críticos durante la fase de integración.

Cuestiones Negativas

1. **Sesgo de las estadísticas de GitHub:** La sensación de que el trabajo se mide por "commits" o líneas de código en lugar de por impacto o dificultad.
2. **Sincronización de tiempos:** Aunque todos cumplieron, los diferentes ritmos a veces generaban cuellos de botella puntuales.
3. **Gestión inicial de la complejidad:** Infravaloramos el tiempo necesario para configurar el entorno de desarrollo inicial.

Justificación y Soluciones

Para el tema de las estadísticas, la solución es este mismo informe: dar peso a la valoración cualitativa sobre la cuantitativa. Para los tiempos, una planificación de *deadlines* internos más estricta podría haber ayudado.

4. Lecciones aprendidas

- **¿Qué cosas hubieseis hecho de otra manera?** Establecer hitos de integración más frecuentes para no dejar el grueso del ensamblado para la fase final.
- **¿Hubiésemos necesitado algo adicional por parte del docente?** Quizás una sesión específica sobre cómo interpretar las métricas de Git de forma más humana y menos matemática, para evitar que los alumnos se sientan juzgados injustamente por números.

5. Justificación de la distribución de esfuerzos (GitHub)

- **¿El esfuerzo es equitativo? Sí.** Consideramos que el esfuerzo real ha sido equivalente en todos los integrantes.
- **Si hay diferencias importantes en GitHub, ¿a qué se deben?** Las estadísticas de GitHub (commits, líneas añadidas/borradas) no reflejan la realidad del trabajo de este equipo. En nuestro caso:
 - Ciertas tareas de backend requieren horas de investigación y lógica que resultan en pocas líneas de código, mientras que tareas de frontend o configuración pueden generar miles de líneas de forma automática o mediante plantillas.
 - Cuando un integrante hace un trabajo excesivo o un "refactor" general, sus estadísticas se disparan, haciendo que el trabajo de los demás parezca menor por comparación, lo cual es falso y desmotivador.
 - **Valoración:** No consideramos que el *github-analyzer* fomente un ambiente positivo; al contrario, resta valor a la colaboración orgánica.
- **Distribución equitativa (sobre 100 puntos):**
 - Integrante 1: 25 pts
 - Integrante 2: 25 pts
 - Integrante 3: 25 pts
 - Integrante 4: 25 pts

6. Uso de herramientas de Inteligencia Artificial Generativa

- **Herramientas utilizadas:** ChatGPT-4, GitHub Copilot y Gemini.
- **Utilidad:** Han sido muy útiles para la depuración de errores (explicar por qué fallaba una función) y para la generación de código repetitivo). También para la traducción de comentarios y documentación.
- **Problemas encontrados:** A veces generan soluciones obsoletas o "alucinaciones" que no encajan con la arquitectura de nuestro proyecto, lo que requiere supervisión constante.
- **Impacto en el aprendizaje:** Facilitan el aprendizaje al actuar como un tutor 24/7 que explica conceptos, pero existe el riesgo de copiar sin entender si no se usan con criterio.
- **Impacto en el futuro profesional:** Serán herramientas estándar. El programador del futuro será más un "arquitecto de soluciones" y un revisor crítico que un mero escritor de código.